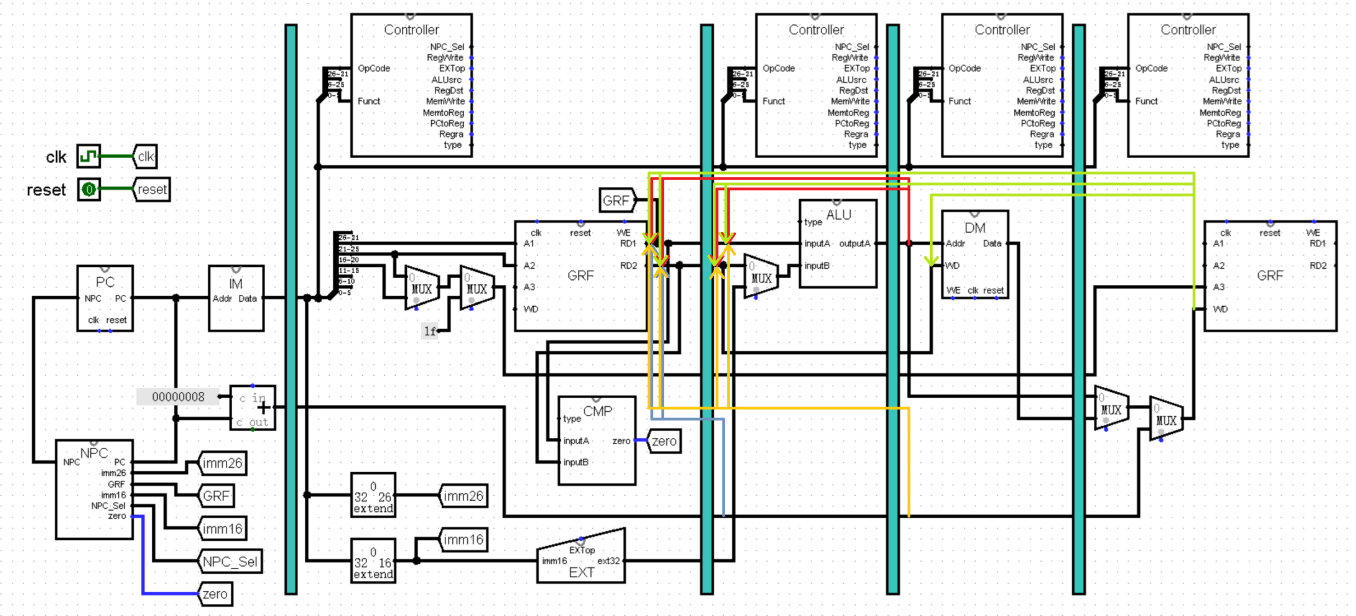
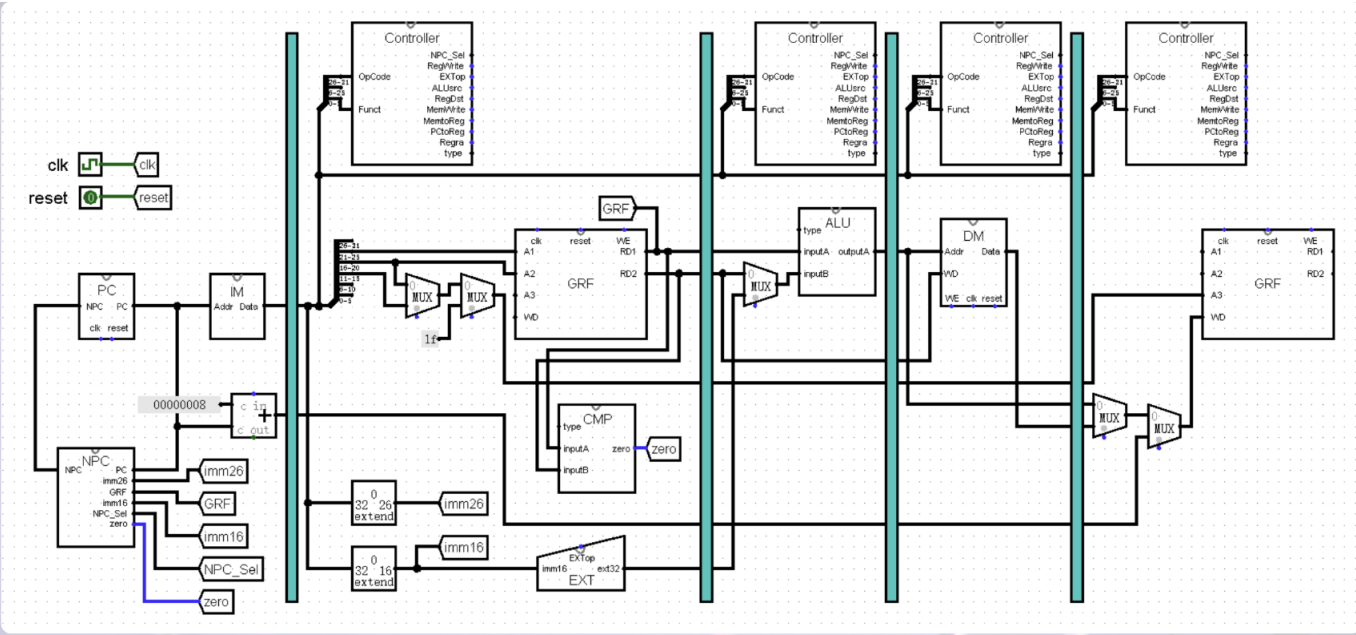
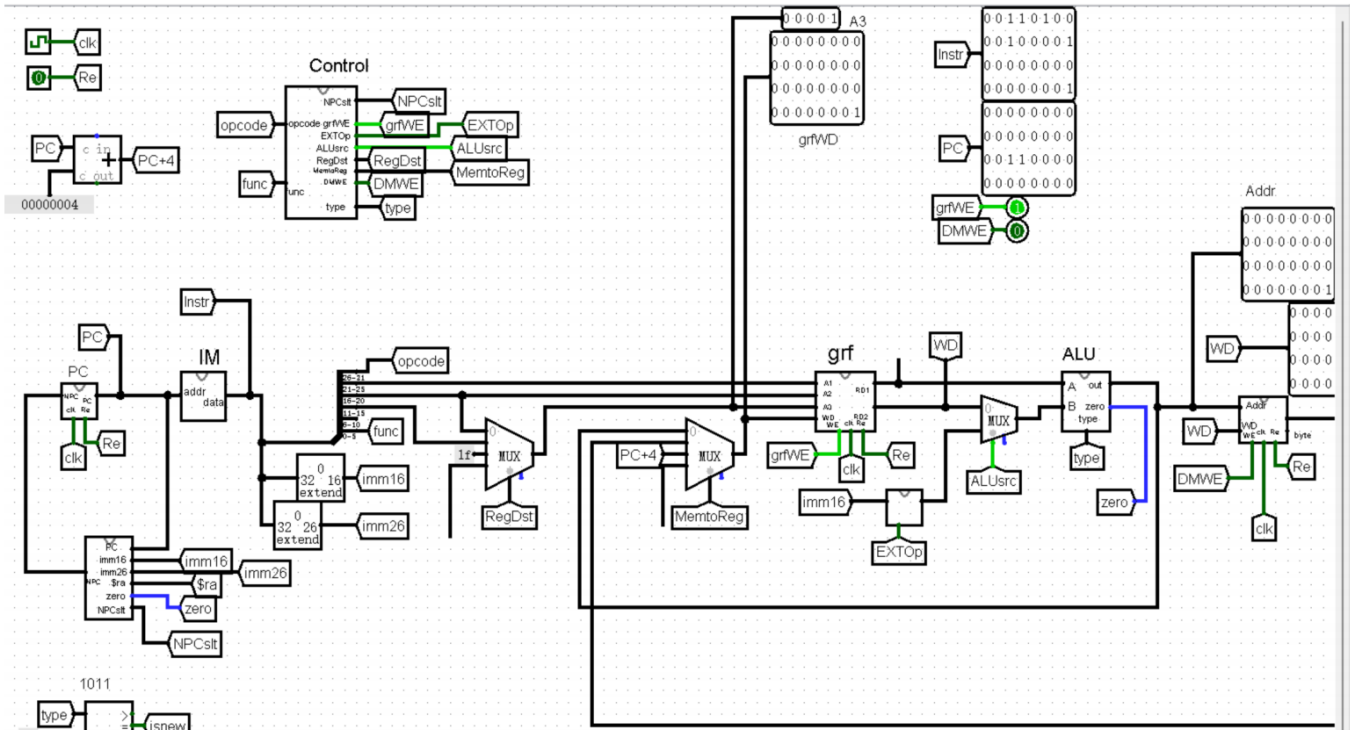


P5设计报告

实现的指令集：{ add, sub, ori, lw, sw, beq, lui, jal, jr, nop } 看到\$31<-PC+8 就要修改isE_PC8 isM_PC8的两条转发通路！！！！



p4cpu :



阻塞过度导致的TLE问题

在DE_stall中分成isnew ! isnew两种情况：

```
assign DE_rs_Stall =(E_type==4'b1011&&D_rs!=5'b0&& 这个约束条件根据题目调整D_rs >=
E_A3这个约束条件根据题目调整 && D_t_rsuse != 4'hf && E_t_new != 4'hf && D_t_rsuse <
E_t_new)
|| (E_type!=4'b1011 ( 直接加这一句就可以了 ) &&E_A3 != 5'b0 && D_rs == E_A3 &&
D_t_rsuse != 4'hf && E_t_new != 4'hf && D_t_rsuse < E_t_new);
```

```
// 如果 E 级指令是 lwtxr 指令，则当 D_rs 不是 0 号寄存器，且 D_t_rsuse 存在，且 D_rs
大于等于 E_rt ( 即 E_A3 ) 时才阻塞
// 需要注意在 E 级指令不是 lwtxr 指令时，在判断条件前加上 E_type != LWTXR，避免被上一
条判断漏下来的 lwtxr 指令进入接下来的判断！
// 如果 M 级指令是 lwtxr 指令，需要使用修正过后的 A3 值
assign D_stall = (E_type == 4'b1011 && D_rs != 5'b00000 && D_t_rsuse != 4'hf &&
D_rs >= E_A3) ? 1'b1 :
(E_type == 4'b1011 && D_rt != 5'b00000 && D_t_rtuse != 4'hf &&
D_rt >= E_A3) ? 1'b1 :
(E_type != 4'b1011 && E_A3 != 5'b00000 && D_rs == E_A3 &&
D_t_rsuse != 4'hf && E_t_new != 4'hf && D_t_rsuse < E_t_new) ? 1'b1 :
(E_type != 4'b1011 && E_A3 != 5'b00000 && D_rt == E_A3 &&
D_t_rtuse != 4'hf && E_t_new != 4'hf && D_t_rtuse < E_t_new) ? 1'b1 :
(M_fixedA3 != 5'b00000 && D_rs == M_fixedA3 && D_t_rsuse != 4'hf
&& M_t_new != 4'hf && D_t_rsuse < M_t_new) ? 1'b1 :
(M_fixedA3 != 5'b00000 && D_rt == M_fixedA3 && D_t_rtuse != 4'hf
&& M_t_new != 4'hf && D_t_rtuse < M_t_new) ? 1'b1 :
(W_A3 != 5'b00000 && D_rs == W_A3 && D_t_rsuse != 4'hf && W_t_new
!= 4'hf && D_t_rsuse < W_t_new) ? 1'b1 :
```

```
(W_A3 != 5'b000000 && D_rt == W_A3 && D_t_rtuse != 4'hf && W_t_new
!= 4'hf && D_t_rtuse < W_t_new) ? 1'b1 : 1'b0;
```

```
.text
ori $1, $0, 1
ori $2, $0, 2
ori $3, $0, 3
ori $4, $0, 4
sw $4,0($0)
```

十五条数据通路解决转发问题

转发 (forward) 判断条件

1. 是否为 isPC8
2. 寄存器编号是否为 0
3. 转发起点和终点寄存器是否相同
4. t_use 值是否存在
5. t_new 值是否存在
6. t_use 是否大于等于 t_new

十五条数据通路

D_RD1_from_M_PC8 : 起点 : D级的RD1 -> 终点 : E级的PC8

D_RD2_from_M_PC8 : 起点 : D级的RD2 -> 终点 : E级的PC8

E_RD1_from_M_PC8 : 起点 : D级的RD1 -> 终点 : M级的PC8

E_RD2_from_M_PC8 : 起点 : D级的RD2 -> 终点 : M级的PC8

D_RD1_from_M : 起点 : D级的RD1 -> 终点 : M级

D_RD2_from_M : 起点 : D级的RD2 -> 终点 : M级

E_RD1_from_M : 起点 : E级的RD1 -> 终点 : M级

E_RD2_from_M : 起点 : E级的RD2 -> 终点 : M级

D_RD1_from_W : 起点 : D级的RD1 -> 终点 : W级

D_RD2_from_W : 起点 : D级的RD2 -> 终点 : W级

E_RD1_from_W : 起点 : E级的RD1 -> 终点 : W级

E_RD2_from_W : 起点 : E级的RD2 -> 终点 : W级

M_RD2_from_W : 起点 : M级的RD2 -> 终点 : W级

阻塞

暂停 (stall) 判断条件

DE DM DW都要比较

1. 寄存器编号是否为 0
2. 转发起点和终点寄存器是否相同
3. t_use 值是否存在
4. t_new 值是否存在
5. t_use 是否小于 t_new

阻塞 (stall)

1. PC冻结
2. D级寄存器冻结
3. E级寄存器清空 (插入nop指令)

```
//P5 test_code
ori $28, $0, 0
ori $29, $0, 0
ori $1, $0, 4112
lui $2, 34595
ori $3, $0, 30806
lui $4, 34303
ori $5, $0, 1
lui $6, 65535
ori $7, $0, 65535
add $1, $1, $2
add $9, $1, $3
sub $8, $1, $2
sub $0, $7, $0
beq $28, $17, label1
nop
beq $0, $0, label2
nop
label1:
beq $1, $2, label2
nop
ori $2, $0, 12
nop
nop
nop
nop
jal label3
sw $1, 0($2)
beq $0, $0, label2
add $1, $1, $2
label3:
add $1, $1, $2
add $1, $1, $2
add $1, $1, $2
sw $31, 0($2)
lw $1, 0($2)
nop
```

```

nop
nop
jr $1
sw $31, 0($2)
label2:
beq $0, $0, label2
nop

```

总述

CPU 为的 5 级流水线阶段为：

阶段	简称	功能概述
取指阶段 (Fetch)	F	从指令存储器中读取指令
译码阶段 (Decode)	D	从寄存器文件中读取源操作数并对指令译码以便得到控制信号
执行阶段 (Execute)	E	使用 ALU 执行计算
存储阶段 (Memory)	M	读或写数据存储器
写回阶段 (Writeback)	W	将结果写回到寄存器文件

模块名

F级

PC

信号名	方向	描述
clk	I	时钟信号
reset	I	异步复位信号
PC_en	I	PC使能信号
NPC[31:0]	I	下一条指令的地址
PC[31:0]	O	目前的指令地址

实现方法：用12位地址、32位数据的ROM作为IM。将PC-0x3000存入储存PC的寄存器中，寄存器中的值可以直接对应ROM要获取指令的地址，从而完成指令输出。

IM

信号名	方向	描述
addr	I	当前PC值
Instr[31:0]	O	取出当前的指令

NPC

信号名	方向	描述
F_PC[31:0]	输入	当前取指阶段的PC值（指令地址）
imm16[15:0]	输入	16位立即数（BEQ指令的地址偏移量，需符号扩展后计算跳转地址）
imm26[25:0]	输入	26位立即数（J/JAL指令的跳转地址段，拼接PC高位生成目标地址）
GRF[31:0]	输入	通用寄存器堆读出的地址（JR指令的跳转目标PC值）
NPCslt[1:0]	输入	下一条PC选择控制信号（来自D_Control模块，控制跳转逻辑）
zero	输入	比较结果标志（来自CMP模块，BEQ指令跳转的判断条件）
NPC[31:0]	输出	下一条指令的PC地址（生成PC+4、分支跳转、直接跳转）

D级

D_Control

信号名	方向	描述
opcode[5:0]	输入	指令操作码
func[5:0]	输入	功能码，用于R型指令细分
type[3:0]	输出	运算/操作类型选择信号
NPCslt[1:0]	输出	下一条PC选择信号，控制PC的更新方式
ALUsrc	输出	ALU操作数B的来源选择（寄存器或立即数扩展）
grfWE	输出	通用寄存器堆写使能信号
RegDst[1:0]	输出	通用寄存器堆写地址选择（rd、rt或31号）
EXTOp	输出	立即数扩展类型选择（符号扩展或零扩展）
MemtoReg[1:0]	输出	寄存器堆写入数据来源选择（ALU结果、内存数据或PC+8等）
DMWE	输出	数据存储器写使能信号
t_rs[3:0]	输出	rs寄存器被使用的时间
t_rt[3:0]	输出	rt寄存器被使用的时间
t[3:0]	输出	产生新数据的时间

CMP

信号名	方向	描述
inputA[31:0]	输入	比较操作数A
inputB[31:0]	输入	比较操作数B
type[3:0]	输入	比较类型

信号名	方向	描述
zero[0]	输出	比较结果标志：BEQ类型下inputA等于inputB时为1，否则为0

EXT

信号名	方向	描述
imm16[15:0]	输入	16位立即数（待扩展数据）
EXTOp	输入	扩展模式选择信号（0=零扩展，1=符号扩展）
EXT32[31:0]	输出	扩展后的32位数据

grf

信号名	方向	描述
clk	I	时钟信号
reset	I	异步复位信号
grfWE	I	写入使能信号
PC[31:0]	I	当前PC值--W级的PC(W_PC)
A1[4:0]	I	指定一个寄存器并将其中数据读出到RD1
A2[4:0]	I	指定一个寄存器并将其中数据读出到RD2
A3[4:0]	I	指定一个寄存器作为写入数据的目标
grfWD[31:0]	I	要写入的数据
RD1[31:0]	O	\$rs
RD2[31:0]	O	\$rt

该模块直接使用P0中搭建的GRF(并新增PC的输入便于输出测试)。

E级

E_Control

ALU

信号名	方向	描述
A[31:0]	输入	操作数A
B[31:0]	输入	操作数B
type[3:0]	输入	操作类型选择信号
out[31:0]	输出	运算结果

M级

M_Control

DM

信号名	方向	描述
clk	输入	时钟信号
reset	输入	异步复位信号
DMWE	输入	数据存储器写使能信号
PC[31:0]	输入	当前指令地址 (用于输出测试) ->W_PC
addr[31:0]	输入	数据存储器访问地址
DMWD[31:0]	输入	数据存储器要写入的数据
data[31:0]	输出	数据存储器读出的数据

W级

W_Control

分布式译码中各级Control的输出信号

级别	信号名
D级	D_NPCslt[1:0]
D级	D_RegDst[1:0]
D级	D_EXTOp
E级	E_type[3:0]
E级	E_ALUsrc
M级	M_type[3:0]
M级	M_DMWE
W级	W_type[3:0]
W级	W_grfWE
W级	W_MemtoReg[1:0]

阻塞和转发

指令与信号值对应表

指令	t_rs	t_rt	t
----	------	------	---

指令	t _{rs}	t _{rt}	t
add	4'h1	4'h1	4'h2
sub	4'h1	4'h1	4'h2
ori	4'h1	4'hf	4'h2
lui	4'hf	4'hf	4'h2
lw	4'h1	4'hf	4'h3
sw	4'h1	4'h2	4'hf
beq	4'h0	4'h0	4'hf
j	4'hf	4'hf	4'hf
jal	4'hf	4'hf	4'h0
jr	4'h0	4'hf	4'hf

对于每一条指令：t_{rs}: rs寄存器被使用时间 t_{rt}: rt寄存器被使用时间 t:新值产生的时间内 ~~x~~用f表示

思考题答案

- Q1：我们使用提前分支判断的方法尽早产生结果来减少因不确定而带来的开销，但实际上这种方法并非总能提高效率，请从流水线冒险的角度思考其原因并给出一个指令序列的例子。A:提前分支判断意味着需要提前获得寄存器中的值，即Tuse=0，这提高了数据冒险出现的可能性。例如：`lw $1,10($1) beq $1,$2,label` 这个序列中beq指令需要阻塞至lw获得结果后再判断，这与不进行提前分支判断的效率还低。
- Q2:因为延迟槽的存在，对于 jal 等需要将指令地址写入寄存器的指令，要写回 PC + 8，请思考为什么这样设计？A:PC+8指向的是延迟槽内的指令的下一条指令，从而保证在jr指令调用时不再次执行延迟槽内指令，而是继续执行下一条指令。
- Q3：我们要求大家所有转发数据都来源于流水寄存器而不能是功能部件（如 DM、ALU），请思考为什么？A:组合电路中的门电路的计算也需要时间。如果在模块的后面开始转发，就有可能出现在一个周期中间甚至是末尾的时候数据发生改变的情况，此时转发终点处的组合逻辑可能来不及计算完下个时钟上升沿就已经到来，导致没能够赶上流水，新产生的数据会出现冲突而表现为x值。
- Q4：我们为什么要使用 GPR 内部转发？该如何实现？A:GPR内部转发可以减少阻塞，因为从E、W级输出的数据存进中间的寄存器仍然需要一个时钟周期，而内部转发则可以直接跳过这一个时钟周期而获得结果。实现方法：构造从E_PC8、M_PC、ALUout、grfWD通向RD1、RD2、ALUinputA、ALUinputB的数据通路，并且按照优先级以多路选择器输出给模块。
- Q5：我们转发时数据的需求者和供给者可能来源于哪些位置？共有哪些转发数据通路？**5个起点 4个终点 15条数据通路** 起点：D级的RD1 -> 终点：E级的PC8
起点：D级的RD2 -> 终点：E级的PC8

起点：D级的RD1 -> 终点：M级的PC8

起点：D级的RD2 -> 终点：M级的PC8

起点：E级的RD1 -> 终点：M级的PC8

起点：E级的RD2 -> 终点：M级的PC8

起点：D级的RD1 -> 终点：M级

起点：D级的RD2 -> 终点：M级

起点：E级的RD1 -> 终点：M级

起点：E级的RD2 -> 终点：M级

起点：D级的RD1 -> 终点：W级

起点：D级的RD2 -> 终点：W级

起点：E级的RD1 -> 终点：W级

起点：E级的RD2 -> 终点：W级

起点：M级的RD2 -> 终点：W级

- Q6：在课上测试时，我们需要你现场实现新的指令，对于这些新的指令，你可能需要在原有的数据通路上做哪些扩展或修改？提示：你可以对指令进行分类，思考每一类指令可能修改或扩展哪些位置。A:运算类指令需要考察其需要的寄存器值，设定对应Tuse，改变ALU内计算模块，最后设置Tnew；存取类指令与上述类似，但需要考虑DM的输入输出，在M级做调整；跳转类指令则需要在NPC模块中进行处理，并设置Tuse = 0
- Q7：7.简要描述你的译码器架构，并思考该架构的优势以及不足。A：采用分布式译码方式，每一级获取op和func后，解析出对应的指令，而后输出对应的控制信号。优势为代码可扩展性强，分布控制也有层次性。缺点则为可读性稍差，不易调试。

选做题：-Q1:在冒险的解决中，我们引入了AT法，如果你有其他的解决方案，请简述你的思路，并给出一段指令序列，简单说明你是如何做到尽力转发的。

- Q2:请详细描述你的测试方案及测试数据构造策略。A:本次测试采用“半随机自动生成 + 手动定向优化”的混合策略，尽力全面验证处理器指令执行的正确性，采用Python随机生成（详见Student_judge）的方法，覆盖功能型指令的逻辑完整性、跳转指令的控制流稳定性。在基础模板的基础上，通过手动修改指令序列，针对性引入数据冒险与控制冒险场景两类指令在数据冒险、控制冒险场景下的处理能力，既保证测试用例的批量生成效率，又精准覆盖高风险的边缘执行场景。

我的测试代码：

```
ori $0, $0, 100
ori $1, $0, 48
ori $2, $0, 12
ori $3, $0, 52
ori $4, $0, 144
ori $5, $0, 68
ori $6, $0, 152
ori $7, $0, 48
ori $8, $0, 100
ori $9, $0, 148
ori $10, $0, 80
ori $11, $0, 60
ori $12, $0, 100
ori $13, $0, 24
ori $14, $0, 72
ori $15, $0, 92
ori $16, $0, 44
ori $17, $0, 140
ori $18, $0, 148
```

```
ori $19, $0, 24
ori $20, $0, 100
ori $21, $0, 108
ori $22, $0, 72
ori $23, $0, 8
ori $24, $0, 144
ori $25, $0, 84
ori $26, $0, 140
ori $27, $0, 16
ori $28, $0, 116
ori $29, $0, 124
ori $30, $0, 64
ori $31, $0, 148
```

```
add $1,$2,$3
ori $3,$4,108
ori $0,$2,80
jal Test1
ori $0,$1,0
Back1:
sub $1,$1,$1
sw $1,148($1)
sw $1,112($0)
lw $2,116($0)
jal Test2
lw $0,144($0)
Back2:
sub $4,$4,$4
sw $4,132($4)
sub $4,$4,$4
ori $4,$4,68
sw $3,-16($4)
sw $0,116($0)
jal Test3
sw $3,16($0)
Back3:
sub $3,$2,$1
lw $0,136($0)
lui $2,112
jal Test4
sub $2,$2,$1
Back4:
sw $1,32($0)
sub $1,$1,$1
lw $2,88($1)
sub $1,$1,$2
jal Test5
lui $0,92
Back5:
sw $1,20($0)
sub $4,$0,$4
sw $0,116($0)
jal Test6
add $3,$1,$3
```

```
Back6:
add $0,$1,$3
lui $3,136
sw $2,8($0)
jal Test7
sub $2,$4,$1
Back7:
sub $1,$1,$1
ori $1,$1,80
sw $1,-76($1)
lw $3,100($0)
lui $2,32
jal Test8
sw $4,104($0)
Back8:
sw $0,44($0)
sw $4,60($0)
sub $0,$0,$1
jal Test9
sub $4,$1,$2
Back9:
lui $2,148
sub $2,$2,$2
ori $2,$2,4
lw $1,0($2)
lw $1,84($0)
jal Test10
sw $4,92($0)
Back10:
lw $0,40($0)
sub $4,$4,$4
sw $0,60($4)
sw $1,64($0)
jal Test11
lw $1,96($0)
Back11:
ori $2,$1,124
sub $4,$4,$4
lw $1,48($4)
lui $3,84
jal Test12
lw $1,4($0)
Back12:
sub $2,$2,$2
ori $2,$2,128
lw $3,-88($2)
sw $1,60($0)
lw $3,28($0)
jal Test13
sw $0,84($0)
Back13:
lw $3,8($0)
add $3,$1,$4
sub $0,$0,$0
```

```
sw $4,96($0)
jal Test14
sub $1,$0,$3
Back14:
add $3,$3,$3
lui $3,0
lw $1,64($0)
jal Test15
ori $0,$3,76
jal End
add $4,$0,$1

Test1:
add $3,$4,$2
add $1,$2,$1
lui $1,0
sub $4,$4,$4
sw $4,72($4)
lui $2,4
sub $1,$1,$1
lw $0,12($1)
add $2,$4,$1
lw $3,24($0)
sub $4,$0,$3
sw $2,88($0)
sub $3,$0,$3
sub $4,$3,$0
sub $4,$4,$4
ori $4,$4,152
sw $3,0($4)
sub $4,$4,$4
ori $4,$4,100
sw $1,-8($4)
beq $1,$3,EndTest1
sub $1,$1,$1
beq $0,$2,EndTest1
sub $0,$4,$4
ori $1,$2,116
lw $0,24($0)
add $1,$2,$1
beq $0,$4,EndTest1
sw $0,36($0)
sub $4,$4,$4
ori $4,$4,132
lw $3,-96($4)
add $1,$3,$2
beq $1,$3,EndTest1
sub $1,$0,$1
beq $2,$0,EndTest1
lw $3,128($0)
lui $4,136
EndTest1:
jr $ra
add $0,$2,$4
```

```
Test2:
sub $5,$5,$5
sub $26,$26,$26
ori $26,$26,10
sub $15,$15,$15
ori $15,$15,1
SubTest2:
add $5,$5,$15
beq $5,$26,EndTest2
add $4,$1,$2
sub $1,$1,$1
lui $1,120
lw $0,0($0)
sub $1,$1,$1
ori $1,$1,64
lw $0,-4($1)
lui $1,44
lw $2,116($0)
ori $1,$1,96
lui $1,84
sub $2,$2,$2
lw $3,112($2)
lw $1,0($0)
sw $1,88($0)
ori $4,$3,84
beq $3,$0,SubTest2
sw $1,48($0)
lw $4,84($0)
lw $3,60($0)
sub $1,$1,$1
ori $1,$1,152
sw $4,-36($1)
sub $4,$4,$2
sub $1,$1,$1
ori $1,$1,56
lw $2,-8($1)
lw $0,60($0)
sw $3,24($0)
sub $2,$2,$2
ori $2,$2,148
lw $2,-44($2)
ori $3,$3,0
sw $2,72($0)
sub $1,$1,$1
ori $1,$1,152
sw $3,-48($1)
sub $4,$4,$4
lw $4,124($4)
EndTest2:
jr $ra
lw $4,152($0)
```

Test3:

```
sub $1,$1,$1
ori $1,$1,132
lw $0,-72($1)
sw $0,108($0)
sw $3,116($0)
lui $0,148
add $0,$4,$4
ori $1,$2,84
add $4,$3,$2
sub $2,$4,$0
sw $0,44($0)
sub $3,$3,$3
lw $0,148($3)
sub $0,$0,$0
sw $4,4($0)
sub $1,$4,$1
sub $3,$1,$1
ori $3,$4,128
lw $3,148($0)
ori $1,$3,44
sub $1,$1,$1
ori $1,$1,92
lw $4,-40($1)
add $4,$0,$1
sub $4,$4,$4
ori $4,$4,140
lw $4,-64($4)
ori $0,$2,116
sw $0,80($0)
sub $4,$2,$0
sub $3,$3,$3
lw $4,112($3)
add $3,$4,$4
ori $3,$0,56
EndTest3:
jr $ra
lui $3,116
```

Test4:

```
sub $10,$10,$10
sub $9,$9,$9
ori $9,$9,10
sub $14,$14,$14
ori $14,$14,1
SubTest4:
add $10,$10,$14
beq $10,$9,EndTest4
sw $2,144($0)
lw $4,20($0)
sub $2,$0,$3
ori $3,$1,104
```

```
lw $0,36($0)
lui $1,96
lw $2,136($0)
add $1,$2,$3
beq $1,$2,SubTest4
sw $3,36($0)
sub $3,$3,$3
ori $3,$3,152
lw $4,-108($3)
beq $3,$4,SubTest4
lw $4,156($0)
sub $0,$0,$0
sw $0,108($0)
sub $4,$3,$2
sub $1,$2,$1
sw $1,20($0)
ori $0,$4,132
sw $1,108($0)
sub $4,$4,$4
sw $1,108($4)
lui $2,80
sub $2,$2,$2
lw $4,16($2)
add $4,$2,$4
add $3,$0,$0
lw $1,76($0)
sw $0,96($0)
sub $1,$1,$1
lw $4,152($1)
ori $0,$2,68
EndTest4:
jr $ra
sub $3,$4,$2
```

```
Test5:
sub $8,$8,$8
sub $25,$25,$25
ori $25,$25,10
sub $29,$29,$29
ori $29,$29,1
SubTest5:
add $8,$8,$29
beq $8,$25,EndTest5
sub $4,$4,$1
lw $2,96($0)
lui $3,28
lw $3,116($0)
sub $1,$1,$1
ori $1,$1,48
sw $3,-4($1)
add $4,$2,$4
sub $1,$3,$3
sub $2,$2,$2
```

```
ori $2,$2,132
lw $4,-112($2)
sub $0,$0,$0
lw $0,148($0)
sub $3,$3,$3
lw $0,12($0)
sub $3,$3,$3
lw $3,136($3)
beq $0,$4,SubTest5
ori $1,$2,0
lui $0,44
beq $1,$2,SubTest5
sub $2,$2,$2
add $1,$0,$0
sw $2,80($0)
lui $0,76
sw $1,88($0)
lui $0,36
sub $1,$2,$2
lw $4,132($0)
beq $0,$3,SubTest5
sw $4,0($0)
sub $2,$2,$2
ori $2,$2,124
sw $0,-80($2)
sw $2,8($0)
lui $1,0
EndTest5:
jr $ra
lui $0,152
```

```
Test6:
sw $3,136($0)
sub $1,$1,$3
ori $1,$3,0
ori $3,$3,136
lw $2,48($0)
lui $2,40
sw $1,108($0)
lw $0,120($0)
sub $0,$0,$0
lw $2,144($0)
add $1,$4,$1
sub $1,$0,$2
sw $4,124($0)
sub $3,$0,$0
beq $4,$4,EndTest6
ori $2,$2,40
beq $1,$0,EndTest6
sub $1,$3,$1
sub $1,$4,$4
sw $0,52($0)
sub $2,$2,$2
```

```
lw $1,24($2)
lui $0,4
ori $1,$0,52
sub $1,$1,$1
ori $1,$1,156
sw $2,-120($1)
lui $1,88
sub $3,$3,$3
sw $1,132($3)
sub $3,$3,$3
sw $0,0($3)
ori $3,$3,136
EndTest6:
jr $ra
ori $4,$1,36
```

```
Test7:
sub $4,$4,$4
sw $4,132($4)
lw $3,56($0)
sub $2,$1,$3
lui $0,48
sub $4,$4,$4
ori $4,$4,148
sw $1,-112($4)
lw $1,132($0)
lui $4,44
lw $4,140($0)
sub $1,$1,$1
ori $1,$1,128
sw $0,-68($1)
lw $3,116($0)
ori $0,$1,84
sub $3,$3,$3
ori $3,$3,36
lw $3,-24($3)
add $4,$1,$3
sub $3,$3,$3
ori $3,$3,140
sw $1,-136($3)
beq $2,$2,EndTest7
lui $0,152
lui $2,148
beq $4,$0,EndTest7
ori $4,$3,72
beq $3,$2,EndTest7
sw $0,0($0)
sub $4,$4,$4
ori $4,$4,120
lw $2,-4($4)
sw $4,44($0)
lui $1,128
sub $0,$0,$0
```

```
sw $0,148($0)
sub $1,$1,$1
sw $3,140($1)
lw $2,128($0)
beq $2,$3,EndTest7
add $3,$4,$4
EndTest7:
jr $ra
sub $1,$0,$0

Test8:
sub $18,$18,$18
sub $23,$23,$23
ori $23,$23,10
sub $17,$17,$17
ori $17,$17,1
SubTest8:
add $18,$18,$17
beq $18,$23,EndTest8
sw $1,8($0)
sub $1,$3,$3
add $0,$2,$3
sub $3,$3,$3
sw $4,16($3)
sub $4,$4,$4
ori $4,$4,136
sw $3,-128($4)
lw $2,104($0)
sw $3,124($0)
sub $4,$4,$4
ori $4,$4,140
lw $2,-116($4)
sub $4,$4,$4
ori $4,$4,128
sw $3,-24($4)
lui $4,24
sub $3,$3,$3
lw $2,128($3)
sub $2,$2,$2
sw $3,12($2)
sw $1,140($0)
sw $4,72($0)
lw $1,76($0)
beq $3,$1,SubTest8
ori $0,$1,124
sw $3,8($0)
lui $3,4
lui $1,12
beq $3,$4,SubTest8
sw $4,156($0)
add $0,$2,$2
ori $4,$1,60
sub $2,$0,$4
```

```
sub $4,$4,$4
ori $4,$4,92
lw $4,-64($4)
lw $3,12($0)
sub $3,$3,$1
EndTest8:
jr $ra
lui $4,24

Test9:
ori $4,$3,24
add $1,$4,$0
sub $0,$0,$0
lw $1,108($0)
sub $0,$0,$0
sw $1,32($0)
sub $4,$4,$4
lw $4,0($4)
lw $3,84($0)
sub $2,$4,$0
add $1,$3,$1
ori $3,$3,140
lui $0,52
beq $3,$4,EndTest9
ori $4,$3,76
lw $0,24($0)
lw $3,24($0)
beq $4,$1,EndTest9
sw $4,20($0)
beq $1,$3,EndTest9
lw $4,100($0)
sub $2,$2,$2
ori $2,$2,32
sw $4,0($2)
add $3,$2,$1
sub $3,$3,$3
sw $2,128($3)
ori $2,$4,36
beq $4,$1,EndTest9
lw $0,144($0)
ori $4,$0,100
sub $1,$2,$4
lw $1,140($0)
sub $1,$1,$1
lw $4,148($1)
sub $3,$3,$3
lw $0,56($3)
EndTest9:
jr $ra
lui $4,132

Test10:
```

```
ori $0,$1,60
lui $3,104
sub $3,$4,$1
ori $4,$2,64
sub $1,$1,$1
ori $1,$1,112
lw $3,-32($1)
ori $0,$0,140
sw $1,92($0)
sub $3,$3,$0
ori $2,$2,88
add $4,$0,$3
sw $3,52($0)
add $2,$2,$4
sub $2,$2,$2
ori $2,$2,68
lw $2,-16($2)
sub $3,$3,$3
sw $2,124($3)
ori $4,$3,12
sw $2,152($0)
sub $4,$3,$1
sub $0,$0,$0
lw $4,8($0)
ori $2,$0,124
sub $4,$4,$4
ori $4,$4,152
sw $4,-104($4)
sub $1,$1,$1
sw $3,144($1)
lui $0,8
sub $4,$4,$4
sw $4,128($4)
beq $3,$2,EndTest10
lui $0,156
sub $3,$0,$4
EndTest10:
jr $ra
sw $2,120($0)
```

```
Test11:
lw $2,44($0)
sub $2,$2,$2
lw $3,0($2)
lui $1,0
ori $2,$0,36
sub $4,$4,$4
lw $3,60($4)
sub $0,$0,$0
sw $1,36($0)
ori $0,$3,56
add $1,$4,$1
beq $3,$4,EndTest11
```

```
lw $3,68($0)
sw $2,108($0)
lui $4,16
beq $2,$2,EndTest11
add $2,$4,$4
lui $2,72
sub $2,$3,$4
lw $0,20($0)
sub $1,$1,$1
ori $1,$1,140
lw $4,-16($1)
ori $1,$4,156
lw $0,24($0)
lw $4,40($0)
sub $4,$4,$1
beq $2,$2,EndTest11
sw $1,148($0)
sub $1,$1,$1
ori $1,$1,104
sw $3,-20($1)
sub $4,$4,$4
lw $3,40($4)
ori $1,$2,132
lui $1,92
EndTest11:
jr $ra
lw $4,4($0)
```

```
Test12:
sub $21,$21,$21
sub $29,$29,$29
ori $29,$29,10
sub $17,$17,$17
ori $17,$17,1
SubTest12:
add $21,$21,$17
beq $21,$29,EndTest12
sw $0,152($0)
sw $1,132($0)
sw $1,52($0)
lui $4,100
ori $4,$2,64
add $2,$1,$0
sw $1,124($0)
add $4,$0,$1
beq $3,$2,SubTest12
sw $0,92($0)
lw $3,52($0)
add $0,$2,$2
lui $4,124
lui $4,148
sub $1,$1,$1
ori $1,$1,64
```

```
sw $2,-40($1)
sw $2,52($0)
lui $3,156
lui $2,80
sw $1,12($0)
add $4,$0,$1
lui $3,72
sw $4,88($0)
sub $3,$4,$0
sub $2,$2,$2
sw $3,4($2)
lui $0,28
sw $1,72($0)
sub $1,$1,$1
ori $1,$1,84
lw $0,-12($1)
EndTest12:
jr $ra
lui $4,92
```

```
Test13:
sub $7,$7,$7
sub $19,$19,$19
ori $19,$19,10
sub $26,$26,$26
ori $26,$26,1
SubTest13:
add $7,$7,$26
beq $7,$19,EndTest13
sub $4,$0,$1
sub $2,$2,$2
lw $3,24($2)
sub $1,$1,$1
sw $2,20($1)
lw $0,4($0)
sw $4,120($0)
lw $3,124($0)
lui $2,28
sub $4,$4,$4
ori $4,$4,136
lw $3,-80($4)
add $3,$1,$4
ori $1,$2,152
sub $4,$4,$4
ori $4,$4,128
lw $3,-72($4)
lui $0,20
beq $3,$2,SubTest13
sw $1,24($0)
sub $4,$4,$4
sw $3,136($4)
sub $4,$4,$4
sw $1,136($4)
```

```
sub $2,$2,$2
lw $1,120($2)
sw $1,88($0)
sub $4,$0,$0
sub $2,$2,$2
ori $2,$2,88
sw $3,-80($2)
sw $1,148($0)
lui $4,84
add $0,$1,$0
lui $0,64
sub $1,$1,$1
sw $4,80($1)
ori $3,$3,132
sub $3,$3,$3
lw $0,112($3)
EndTest13:
jr $ra
lui $0,148
```

```
Test14:
sub $24,$24,$24
sub $5,$5,$5
ori $5,$5,10
sub $19,$19,$19
ori $19,$19,1
SubTest14:
add $24,$24,$19
beq $24,$5,EndTest14
lw $3,48($0)
ori $4,$1,60
sw $2,20($0)
lw $3,64($0)
lui $0,48
add $2,$0,$0
lw $2,36($0)
add $1,$0,$2
ori $0,$4,104
beq $3,$4,SubTest14
add $4,$0,$2
lw $2,148($0)
sub $1,$1,$1
ori $1,$1,88
lw $1,-80($1)
sub $1,$1,$1
ori $1,$1,148
lw $4,-16($1)
sub $0,$2,$2
sub $2,$2,$2
ori $2,$2,80
sw $0,-72($2)
beq $4,$2,SubTest14
sw $0,124($0)
```

```
sub $0,$0,$0
lw $0,124($0)
add $4,$4,$4
sw $1,44($0)
sub $3,$2,$4
add $3,$1,$4
sub $4,$4,$4
sw $4,104($4)
sw $1,16($0)
beq $4,$3,SubTest14
sw $1,112($0)
sub $1,$1,$1
ori $1,$1,48
lw $4,0($1)
beq $1,$2,SubTest14
sub $0,$3,$0
EndTest14:
jr $ra
sw $3,40($0)
```

```
Test15:
sub $15,$15,$15
sub $29,$29,$29
ori $29,$29,10
sub $6,$6,$6
ori $6,$6,1
SubTest15:
add $15,$15,$6
beq $15,$29,EndTest15
add $0,$2,$1
lui $1,96
sub $4,$2,$2
sub $1,$2,$1
ori $4,$4,32
sw $1,20($0)
add $1,$3,$2
sub $0,$0,$0
sw $0,128($0)
sw $2,104($0)
add $0,$1,$2
beq $2,$3,SubTest15
lw $4,32($0)
lw $1,48($0)
sw $3,108($0)
sub $4,$0,$4
sub $3,$3,$3
ori $3,$3,144
lw $4,-76($3)
ori $3,$1,80
sub $1,$2,$4
sw $4,36($0)
lui $1,100
beq $3,$0,SubTest15
```

```
lui $2,140
add $0,$0,$2
sub $3,$3,$3
ori $3,$3,92
lw $0,-80($3)
lui $2,76
lui $3,12
lw $4,56($0)
ori $2,$3,36
EndTest15:
jr $ra
sub $3,$4,$2
End:
```